

Q 1:

```
int main() {
    int n;
```

```
    cout << "Enter num of locations: ";
    cin >> n;
```

```
    int a[n];
    cout << "Enter the cable lengths";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
```

```
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
```

```
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
```

```
        while (j >= 0 && a[j] > key) {
            a[j+1] = a[j];
            j = j - 1;
        }
```

```
        a[j+1] = key;
```

```
    for (int k = 0; k < n; k++) cout << a[k] << " ";
    cout << endl;
}
```

Teacher's Signature \_\_\_\_\_

Elite PAPER PRODUCTS

Page No.

Q2:

```
int main() {
```

```
    int n, pos, newCap;
```

```
    cout << "Enter num of shelves";
```

```
    cin >> n;
```

```
    int *shelves = new int [n];
```

```
    cout << "Capacity of each shelf";
```

```
    for (int i=0; i<n; i++) cin << shelves[i];
```

```
    cout << "Enter position to add to the new shelf:";
```

```
    cin >> pos;
```

```
    cout << "Enter capacity of new shelf";
```

```
    cin >> newCap;
```

```
    cout << "OG shelf capacity";
```

```
    for (int i=0; i<n; i++) cout << shelves[i];
```

```
    int *newShelves = new int [n+1];
```

```
    for (int i=0; i<=n; i++) {
```

```
        if (i == pos) newShelves[i] = newCap;
```

```
        else if (i < pos) newShelves[i] = shelves[i];
```

```
        else newShelves[i] = shelves[i-1];
```

```
    }
```

```
    delete [] shelves;
```

```
for (int i = 1; i <= n; i++) {  
    int key = newShelves[i], j = i - 1;  
    while (j >= 0 && newShelves[j] > key) {  
        newShelves[j + 1] = newShelves[j];  
        j--;  
    }  
    newShelves[j + 1] = key;  
}
```

```
for (int i = 0; i <= n; i++) cout << newShelves[i];  
delete [] newShelves;  
}
```

Q3.

Date \_\_\_\_\_ 20 \_\_\_\_\_

```
int main() {  
    int arr[8];  
    int gap=8, pass_min=1;  
    bool sorted=false;  
  
    cout << "Enter 8 package weights";  
    for(int i=0; i<8; i++) cin << arr[i];  
  
    for(int i=0; i<8; i++) cout << arr[i];
```

```
while ( gap > 1 || !sorted) {
```

```
    gap = (int)(gap/1.3);  
    if (gap < 1) gap = 1;
```

```
    sorted = true;
```

```
    for(int i=0; i+gap<8; i++) {  
        if (arr[i] > arr[i+gap]) {  
            swap(arr[i], arr[i+gap]);  
            sorted = false;  
        }  
    }
```

```
}
```

```
for(int i=0; i<8; i++) cout << arr[i];  
cout << endl;
```

```
}
```

```
}
```

## Question 4

```
struct Rider {
```

```
    int id, order;
```

```
    string name;
```

```
    Rider * next;
```

```
Rider (int i, string n, int o) : id(i), name(n),  
    order(o), next(NULL) {}
```

```
};
```

```
class KFC {
```

```
    Rider * head = nullptr;
```

```
    int count () {
```

```
        if (!head) return 0;
```

```
        int c = 0; Rider * t = head;
```

```
        do {
```

```
            c++;
```

```
            t = t->next;
```

```
        } while (t != head)
```

```
        return c;
```

```
    }
```

```
void insBeg (int i, string n, int o) {
```

```
    Rider * nn = new Rider (i, n, o);
```

```
    if (!head) {
```

```
        head = nn;
```

```
        nn->next = head;
```

```
        return;
```

```
    }
```

```
    Rider * t = head;
```

```

1 while (t → next != head) t = t → next;
   nn → next = head; t → next = nn; head = nn;
}

```

```

void insEnd (int i, string n, int o) {

```

```

    Rides * nn = new Rides (i, n, o);
    if (!head) {
        head = nn;
        nn → next = head;
        return;
    }

```

```

    Rides * t = head;
    while (t → next != head) t = t → next;
    t → next = nn; nn → next = head;
}

```

```

void insPos (int i, string n, int o, int p) {
    int len = count ();
    if (p < 1 || p > len + 1) return;
    if (p == 1) {
        insBeg (i, n, o); return;
    }

```

```

    Rides * nn = new Rides (i, n, o), *t = head;
    for (int idx = 1; idx < p - 1; idx++) t = t → next;
    nn → next = t → next; t → next = nn;
}

```

```

void delBeg() {
    if (!head) return;
    Rider * t = head;
    if (head->next == head) {
        delete head;
        head = NULL;
        return;
    }

```

```

    while (t->next != head) t = t->next;
    Rider * del = head;
    head = head->next;
    t->next = head;
    delete del;
}

```

```

void delEnd() {
    if (!head) return;
    if (head->next == head) {
        delete head;
        head = NULL;
        return;
    }

```

```

    Rider * t = head, *p = NULL;
    while (t->next != head) {
        p = t;
        t = t->next;
    }

```

```

    p->next = head; delete t;
}

```

```

void delPos (int p) {
    int len = count ();
    if (p < 1 || p > len) return;
    if (p == 1) { delBeg (); return; }
    Rider* t = head, * pNode = nullptr;
    for (int idx = 1; idx < p; idx++) {
        pNode = t;
        t = t -> next;
    }
    pNode -> next = t -> next;
    delete t;
}

```

```

Rider* search (int i) {
    if (!head) return nullptr;
    Rider* t = head;
    do {
        if (t -> id == i) return t;
        t = t -> next;
    } while (t != head);
    return null;
}

```

```

void update (int i, string, int o) {
    Rider* r = search (i);
    if (r) { r -> name; r -> orders = o; }
}

```

```

void display () {
    if (!head) return;
    Rider* t = head;
    do {
        cout << t->id << " " << t->name << " " << t->orders;
        t = t->next;
    } while (t != head);
}

```

```

void traverse from (int i) {
    rider* start = search (i);
    if (!start) return;
    Rider* t = start;
    do {
        cout << t->id << " -> ";
        t = t->next;
    } while (t != head);
}

```

```

int main () {
    KFCirculars sys;
    sys.insEnd (101, "Rider A", 5);
    sys.insBeg (100, "Rider B", 2);
    sys.insPos (102, "Rider C", 4, 2);
    cout << sys.count ();
    sys.display ();
    sys.update (102, "Rider C - updated", 9);
    sys.traverse from (102);
    sys.delPos (2);
    sys.delEnd ();
    return 0;
}

```

Teacher's Signature \_\_\_\_\_

Q5:

struct Node {

int id; string name, cat;

double price; Node\* next;

Node (int i, string n, string c, double p):

id(i), name(n), cat(c), price(p),

next(NULL) {}

};

void insert (Node\* &head, int id, string n,  
string c, double p) {

Node\* nn = new Node (id, n, c, p);

if (!head || head-&gt;id &gt;= id) {

nn-&gt;next = head;

head = nn; return; }

Node\* curr = head;

while (curr-&gt;next &amp;&amp; curr-&gt;next-&gt;id &lt; id)

curr = curr-&gt;next;

nn-&gt;next = curr-&gt;next; curr-&gt;next = nn;

}

void print (Node\* curr) {

while (curr) {

cout &lt;&lt; curr-&gt;id &lt;&lt; " " &lt;&lt; curr-&gt;name &lt;&lt;

curr-&gt;cat &lt;&lt; " " &lt;&lt; curr-&gt;price &lt;&lt; endl;

curr = curr-&gt;next;

}

}

Node \* merge (Node \* l1, Node \* l2) {

Node dummy (0, "", "", 0), \* tail = &dummy;

while (l1 && l2) {

if (l1->id < l2->id) {

tail->next = l1;

l1 = l1->next;

}

else if (l2->id < l1->id) {

tail->next = l2;

l2 = l2->next;

}

else {

Node \* next1 = l1->next, \* next2 = l2->next;

if (l1->price < l2->price) {

tail->next = l1;

delete l2;

}

else { tail->next = l2; delete l1; }

l1 = next1; l2 = next2;

}

tail = tail->next;

}

tail->next = l1 ? l1 : l2;

return dummy->next;

}

```
int main() {
```

```
    Node *gc = NULL, *bh = NULL;
    insert(gc, 103, "Moisturizer", "Skin", 25.5);
    insert(gc, 101, "Sunscreen", "Skin", 19.9);
    insert(gc, 104, "Foundation", "Make", 35.0);
    insert(bh, 102, "Cleanser", "Skin", 12.4);
    insert(bh, 104, "Foundation", "Make", 30.0);
```

```
    cout << "Glow Care: \n";
    print(gc);
    cout << "Beauty hub: \n";
    print(bh);
```

```
    Node* master = merge(gc, bh);
    cout << "Merged Master: \n";
    print(master);
```

```
    int count = 0;    double total = 0;
    while (master) {
        count++;
        total += master->price;
        Node* t = master;
        master = master->next;
        delete t;
    }
```

```
    cout << count << total;
```

Q6

```

struct Node {
    string t; int g; int d; bool exp;
    Node* next;
    Node(string title, int genre, int dur, bool e):
        t(title), g(genre), d(dur), exp(e), next(NULL) {}
};

```

```

class PlaylistManager {
    Node* head = NULL;
    int opCount = 0;

    Node* get(int p) {
        Node* c = head;
        while (c && p-- > 0) c = c->next;
        return c;
    }
}

```

```

int size() { int c = 0;
    for (Node* curr = head; curr; curr = curr->next)
        c++;
}

```

```

bool invalid(int p, int q, int d) {
    Node* p1 = get(p-1), *p2 = get(p-2),
        *n1 = get(p), *n2 = get(p+1);
}

```

```
if ((p1 && p1->g == g) || (n1 && n1->g == g))
    return true;
```

```
if (p2 && p1 && p2->d + p1->d > 600)
    return true;
```

```
if (p1 && p1 && p->d + d + n1->d > 600)
    return true;
```

```
if (n1 && n2 && d + n1->d + n2->d > 600)
    return true;
```

```
return false;
```

```
}
```

```
void ankit () {
```

```
    if (++opCount % 3 != 0) return;
    cout << "System Check";
```

```
for (Node* c = head; c; c = c->next) {
```

```
    if ((c->next && c->g == c->next->g) ||
```

```
        (c->next && c->next->next &&
```

```
        c->d + c->next->d + c->next->next->d > 600))
```

```
    { cout << "Conflict found";
```

```
        return; }
```

```
}
```

```
cout << "All rules passed. \n";
```

```
}
```

public:

```

void insert (string t, int g, int d, bool exp,
             int p) {
    int len = size();
    p = max(0, min(p, len));
    while (pc = len & g, invariant(p, g, d)) p++;
    if (p > len) p = len;
    Node* n = new Node(t, g, d, exp);
    if (p == 0) { n->next = head; head = n; }
    else {
        Node* prev = get(p-1);
        n->next = prev->next;
        prev->next = n;
    }
    cout << "Inserted " << t << " at " << p << "\n";
    ansLit();
}

```

```

void remove (int p) {
    Node* t = get(p);
    if (!t || (t->exp & g, cout << "Error: Cannot delete
    explicit song.\n", 1))) return;
    if (p == 0) { head = head->next; delete t; }
    else {
        Node* prev = get(p-1);
        prev->next = t->next;
        delete t;
    }
    cout << "Deleted song at " << p << "\n";
    ansLit();
}

```

```

void search (string title) {
    int idx = 0;
    for (Node* c = head; c = c->next, idx++; )
        if (c->t == title) {
            cout << "Found" << title << " at " << idx << " \n";
            anslit(1);
            return;
        }
    cout << "Not found.\n"; anslit(n);
}

```

```

void setExplicit (int p, bool exp) {
    if (Node* t = get(p)) t->exp = exp;
}

```

```

int main () {
    PlaylistManages pm;
    pm.insert ("Song A", 0, 180, false, 0);
    pm.insert ("Song B", 0, 200, false, 1);
    pm.search ("Song A");
    pm.insert ("Song C", 0, 190, true, 1);
    pm.remove (2);
    pm.setExplicit (2, false);
    pm.remove (2);
    return 0;
}

```

Q-7

Date \_\_\_\_\_ 20 \_\_\_\_\_

```
struct Combatant {  
    string name;  
    int hp, atk;  
};
```

```
struct Node {  
    Combatant data;  
    Node *prev = nullptr, *next = NULL;  
};
```

```
struct team {  
    Node *head = NULL, *tail = NULL;  
    int size = 0;
```

```
void add(string name) {  
    Node *n = new Node { {name, rand() % 70 + 1,  
        rand() % 5 + 1} };  
    if (!head) head = tail = n;  
    else {  
        tail->next = n;  
        n->prev = tail;  
        tail = n;  
    }  
    size++;  
}
```

Teacher's Signature \_\_\_\_\_

Elite PAPER PRODUCTS

Page No.

```

void remove (Node *n) {
    if (!n) return;
    if (n == head) head = head->next;
    if (n == tail) tail = tail->prev;
    if (n->prev) n->prev->next = n->next;
    if (n->next) n->next->prev = n->prev;
    delete n;
    size--;
}

```

```

Node * find (string in) {
    Node * curr = head;
    int idx = 1;
    try {
        int pos = stoi(in);
        while (curr & idx != pos) {
            curr = curr->next;
            idx++;
        }
        if (!curr) return curr;
    } catch (...) {}
    curr = head;
    while (curr) {
        if (curr->data.name == in) return curr;
        curr = curr->next;
    }
    return NULL;
}

```

```

Node* getRandom () {
    if (!head) return NULL;
    int r = rand() % size;
    Node* curr = head;
    while (r-- > 0) curr = curr->next;
    return curr;
}

```

```

void showNames () {
    Node* curr = head; int i = 1;
    while (curr) {
        cout << i << " " << curr->data.name <<
            (curr->next ? ", " : "");
        curr = curr->next;
    }
    cout << "\n";
}

```

```

void showStats () {
    Node* curr = head;
    while (curr) {
        cout << " - " << curr->data.name << " IHP: " <<
            curr->data.hp << " ATK: " << curr->data.atk <<
            "\n";
        curr = curr->next;
    }
}

```

```

int totalHP () {
    int t = 0; Node* curr = head;
    while (curr) { t += curr->data.hp; curr = curr->next; }
    return t;
}

```

Teacher's Signature \_\_\_\_\_

```

int main() {
    srand(time(0));
    Team heroes, enemies;
    string h[] = {"knight", "Mage", "Rogue", "Cleric",
                 "Paladin"};
    string e[] = {"Goblin", "Orc", "Troll", "Dragon",
                 "Wraith"};
    for (int i = 0; i < 5; i++) {
        heroes.add(h[i]);
        enemies.add(e[i]);
    }

    int round = 1;
    while (round <= 10 || (heroes.size() > 0 && enemies.size() > 0)) {
        if (heroes.size() == 0 || enemies.size() == 0) break;

        cout << "Round" << round << "Heroes";
        heroes.showName();
        cout << "Enemies";
        enemies.showName();

        Node *atk = NULL, *tar = NULL;
        string choice;
        while (!atk) {
            cout << "Select Attacker:";
            cin >> choice;
            atk = heroes.find(choice);
        }
    }
}

```

```

while (!tar) {
    cout << "Select target:";
    cin >> choice;
    tar = enemies.find(choice);
}

```

```

int dmg = atk -> data.atk * (rand() / 3 + 1);
tar -> data.hp -= dmg;
cout << ">>" << atk -> data.name << "hits" <<
    tar -> data.name << "for" << dmg << "dmg!\n";
if (tar -> data.hp <= 0) {
    cout << ">>" << tar -> data.name << "died!\n";
    enemies.remove(tar);
}
if (enemies.size == 0) break;

```

```

Node *eAtk = enemies.getRandom(),
    *eTar = heroes.getRandom();
dmg = eAtk -> data.atk * (rand() / 3 + 1);
eTar -> data.hp -= dmg;
cout << ">>" << eAtk -> data.name << "computer hits"
    << eTar -> data.name << "for" << dmg << "dmg!\n";
if (eTar -> data.hp <= 0) {
    cout << ">>" << "Your ally" << eTar -> data.name
        << "fell!\n";
    heroes.remove(eTar);
}

```

```

    round++;
}

```

```
cout << "Game Over";  
hHP = heroes..totalHP(), eHP = enemies..totalHP();  
if (heroes.size == 0) {  
    cout << "Enemies Win! \n";  
    enemies.showStats();  
}  
else if (enemies.size == 0) {  
    cout << "Heroes Win! \n";  
    heroes.showStats();  
}  
else if (hHP > eHP) {  
    cout << "heroes win (higher HP)! \n";  
    heroes.showStats();  
}  
else if (eHP > hHP) {  
    cout << "Enemies win (higher HP)! \n";  
    enemies.showStats();  
}  
else { cout << "It's a draw";  
    heroes.showStats();  
    enemies.showStats();  
}  
}
```

Q8

Date \_\_\_\_\_

20 \_\_\_\_\_

```
struct Node {  
    char movement;  
    int PrevPos;  
    Node * next;  
};
```

```
class MovementStack {  
private:  
    Node * topNode;  
    int size;  
public:  
    MovementStack() {  
        topNode = NULL;  
        size = 0;  
    }
```

```
void push (char move, int currentPos) {  
    Node* newNode = newNode (move, currentPos,  
                             topNode);  
    topNode = newNode;  
    size ++;  
}
```

```
int pop() {  
    if (isEmpty()) return -1;  
    Node* temp = topNode;  
    int oldPos = temp->prevPos;  
    topNode = topNode->next;  
    delete temp;  
    size --;
```

Teacher's Signature \_\_\_\_\_

Elite PAPER PRODUCTS

Page No.

```
return oldPos;
```

```
{
```

```
    bool isEmpty() {
```

```
        return topNode == NULL;
```

```
}
```

```
    bool int getSize() { return size; }
```

```
void display() {
```

```
    if (isEmpty()) {
```

```
        cout << "Empty" << endl;
```

```
        return;
```

```
    }
```

```
Node * curr = topNode;
```

```
while (curr != NULL)
```

```
    cout << curr->movement << " " << curr->next << " " << endl;
```

```
    curr = curr->next;
```

```
}
```

```
{
```

```
~ MovementStack() { while (!isEmpty()) pop(); }
```

```
};
```

Q9.

```

struct Queue {
    int id[100], front = 0, rear = 0;
    void push(int x) { id[rear++] = x; }
    int pop() { return id[front++]; }
    bool empty() { return front == rear; }
    int size() { return rear - front; }
};

```

```

int main() {
    Queue q1, q2, q3;
    int treated[300], totalTreated = 0;
    string cmd;

```

```

    while (cin >> cmd && cmd != "Exit") {
        if (cmd == "Arrive") {
            int id, sec;
            cin >> id >> sec;
            if (sec == 1) q1.push(id);
            else if (sec == 2) q2.push(id);
            else if (sec == 3) q3.push(id);
        } else if (cmd == "Treat") {
            if (!q1.empty()) treated[totalTreated++] = q1.pop();
            else if (!q2.empty()) treated[totalTreated++] = q2.pop();
            else if (!q3.empty()) treated[totalTreated++] = q3.pop();
        }
        cout << "Treatments Order:"
        for (int i = 0; i < totalTreated; i++) cout << treated[i] << " ";
    }
}

```

```
conf << "Total Tracked: << totalTracked
    << "Total Remaining: << (q1.size(), q2.size(), q3.size())
    << "Critical Waiting" << q1.size();
    << "Service waiting" << q2.size();
    << "Normal Waiting" << q3.size();
```

const int MAX\_PLAYERS = 1000;  
 const int MAX\_ACTIONS = 1000;

struct ActionStack {

char data[MAX\_ACTIONS];

int topIdx = -1;

void push(char action) {

if (topIdx < MAX\_ACTIONS - 1) {

data[topIdx++] = action;

}

char pop() {

if (topIdx >= 0) {

return data[topIdx--];

}

return '\0';

int isFull() { return topIdx + 1; }

}

struct Player {

int id, energy = 100; ActionStack actStack;

};

class PlayerQueue {

Player arr[MAX\_PLAYERS];

int frontIdx = 0, rearIdx = 0, count = 0;

public:

```

void enqueue(Players p) {
    if (count < MAX_PLAYERS) {
        arr[rearIdx] = p;
        rearIdx = (rearIdx + 1) % MAX_PLAYERS;
        count++;
    }
}

Players dequeue() {
    Players p = arr[frontIdx];
    frontIdx = (frontIdx + 1) % MAX_PLAYERS;
    count--;
    return p;
}

bool is () const { return count == 0; }
int size () const { return count; }

int main() {
    PlayersQueue waitingLine;
    int totalPlayers;
    cin >> totalPlayers;
    for (int i = 0; i < totalPlayers; i++) {
        Players p;
        cin >> p.id;
        waitingLine.enqueue(p);
    }

    int eliminationIdx = [MAX_PLAYERS], eliminationCount = 0;
    string command;
    while (cin >> command && command != "END" &&
        !waitingLine.isEmpty()) {
        char act = command[0];
        Players currentPlayer = waitingLine.dequeue();
        if (act == 'E')
            currentPlayer.energy = -10;
    }
}

```